

Analysis

This recursive method makes one call per year, and each call does just one multiplication before passing the result to the next call. So if we're forecasting for n years, we get n calls — that makes the time complexity **$O(n)$** . Space is also $O(n)$, because none of the calls finish until the last year is reached, so all of them sit on the call stack at the same time waiting to return.

To optimize it, the simplest fix is to just use a loop instead of recursion. Since each year only depends on the value from the year right before it, there's no real need to keep all those calls stacked up in memory — a loop does the same job in $O(n)$ time but with $O(1)$ space, and there's no risk of stack overflow even for very long forecasts.

An even better option is to skip the step-by-step growth entirely and use the compound growth formula directly: `presentValue * Math.pow(1 + rate, years)`. This gives the answer in one calculation — $O(1)$ time and space — so for a real forecasting tool, this would be the most efficient approach. The recursive version is still useful for understanding the concept, just not ideal for actual production use.